GROEIPROJECT
MODULE 5: TESTEN EN LOGGING

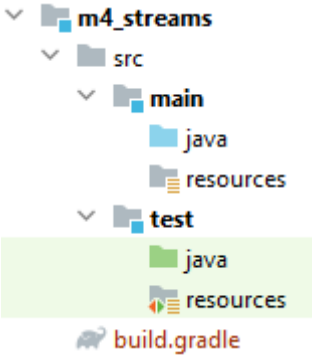


1. VOORBEREIDING

1.1. Maak voorlopig nog **GEEN NIEUWE** module! We gaan eerst testen toevoegen aan de vorige module; `m4_streams`

1.2. Gradle maakt normaal **zelf** een test-folder aan onder de src-folder. Daaronder zit een groen mapje "java" (Groen wil zeggen: "gemarkeerd als Test Sources Root")

1.3. Controleer de `build.gradle` of JUnit 5 bij dependencies vermeld staat. Laat wel de application plugin en mainClassName staan (je wil ook nog altijd de main methode kunnen gebruiken).



2. TESTEN MET JUNIT

2.1. Maak een testklasse `XxxTest` voor je **basisklasse**; in ons geval `DictatorTest` :

a. Ga in de broncode van de basisklasse op de klassenaam staan (bij ons Dictator),druk ALT+ENTER en selecteer *Create Test*

b. Kies JUnit5 als Testing Library.Verander de naam van de klasse naar `DictatorTest`. Verifieer de destination package: `DictatorTest` moet dezelfde package-structuur hebben als de oorspronkelijke klasse `Dictator`. Laat de andere opties voorlopig open en bevestig met "OK"

c. Ga naar de gecreëerde testklasse en voorzie daar minstens twee testobjecten van je basisklasse als private attributen.

d. Druk ALT+INS in de test klasse en kies *SetUp Method.* Gebruik de gegenereerde `@BeforeEach` methode om de private testobjecten (2.1.c) te initialiseren met bepaalde startwaarden.

e. Schrijf een methode `testEquals`, daar test je de `equals` methode van je basisklasse uit als volgt:

1. Controleer of beide testobjecten verschillen van elkaar. Voeg een foutboodschap toe aan je test.
2. Maak nu een lokaal object van de basisklasse aan waarvan de bepalende eigenschappen gelijk zijn aan die van het eerste testobject, maar andere eigenschappen verschillen. Controleer nu of equals true geeft. Vermeld in je foutboodschap welke eigenschappen gelijk zouden moeten zijn en de effectieve waarden in beide objecten.
3. Test uit dat de foutboodschappen gegeven worden als het moet. Zorg dan dat de test slaagt.

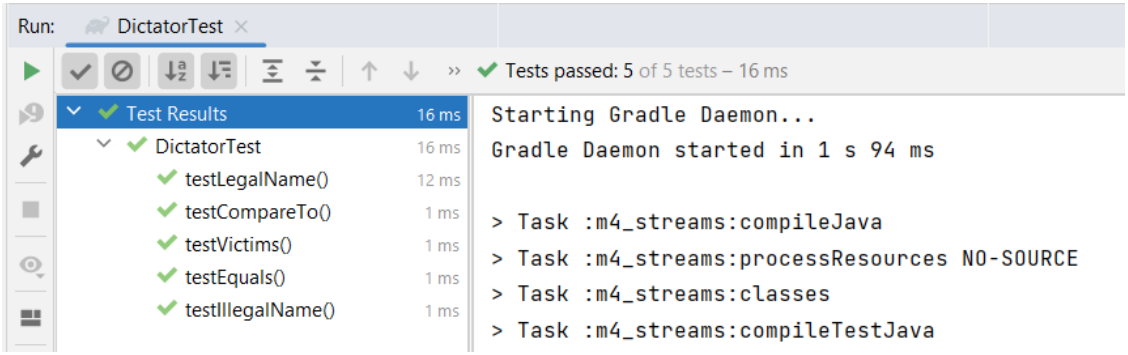
f. Schrijf een methode `testIllegalXxx` (bijvoorbeeld: `testIllegalName`), waarin je een ongeldige waarde aan een setter van de basisklasse meegeeft en dus een `IllegalArgumentException` verwacht. . Maak daarbij gebruik van `assertThrows`. Controleer of de test faalt bij een ongeldige waarde. Zorg daarna dat de test slaagt.

g. Schrijf een nieuwe methode `testLegalXxx` (bijvoorbeeld `testLegalName`), waarin je met assertDoesNotThrow nagaat of de setter bij een correcte waarde GEEN `IllegalArgumentException` gooit. Voorzie ook hier een gepaste foutboodschap.

h. Schrijf een methode `testCompareTo` waarin je de returnwaarde van `compareTo` uittest op de twee in setup geïnitialiseerde attributen. Maak ook een nieuw lokaal object van de basisklasse dat gelijk zou moeten zijn aan één van de testobjecten en test of `compareTo` effectief `0` teruggeeft..

i. Schrijf een testmethode waarin je de waarde van een double-attribuut test. In ons geval wordt dat: `testVictims`. Gebruik daarin `assertEquals` met als parameter een **delta**-waarde.

j. Run en controleer of je groen licht krijgt voor de **vijf** testen:



2.2. Maak ook een testklasse voor je **multiklasse;** in ons geval `Dictator`**`s`**`Test`:

a. Voorzie **vijf** testobjecten van de basisklasse en één multiobject als private attributen.

b. Maak een `@BeforeEach` methode om de 5 basisobjecten te vullen met testgegevens en vervolgens aan het multiobject toe te voegen.

c. Voorzie een methode om het toevoegen uit te testen. Check zeker ook op dubbels!

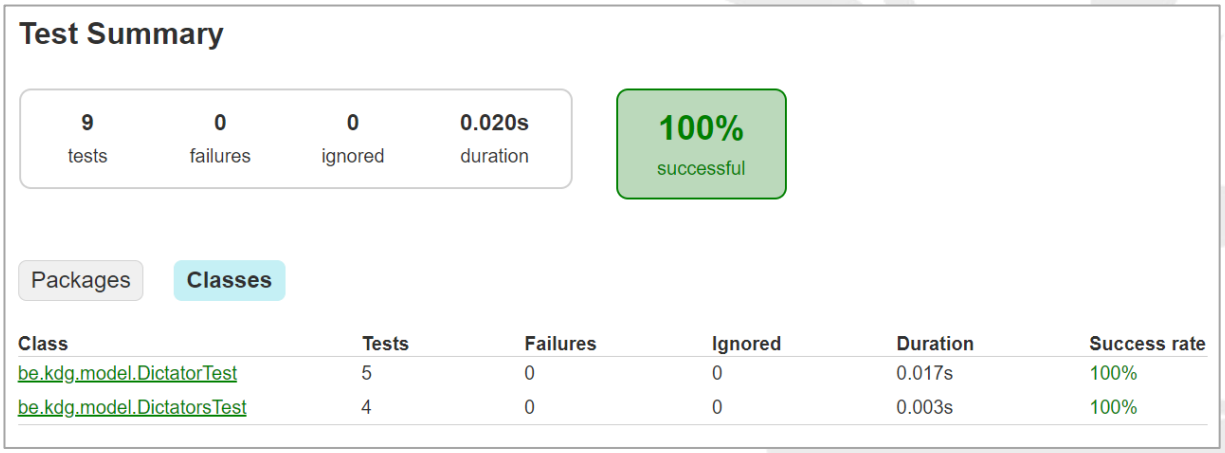
d. Voorzie een methode om het verwijderen uit te proberen. Doe dat door **voor** en **na** de size te checken. Test zeker ook het verwijderen van een niet-bestaand object!

e. Voorzie een methode om het sorteren te testen. Gebruik `assertAll` om een 3-tal posities te controleren.

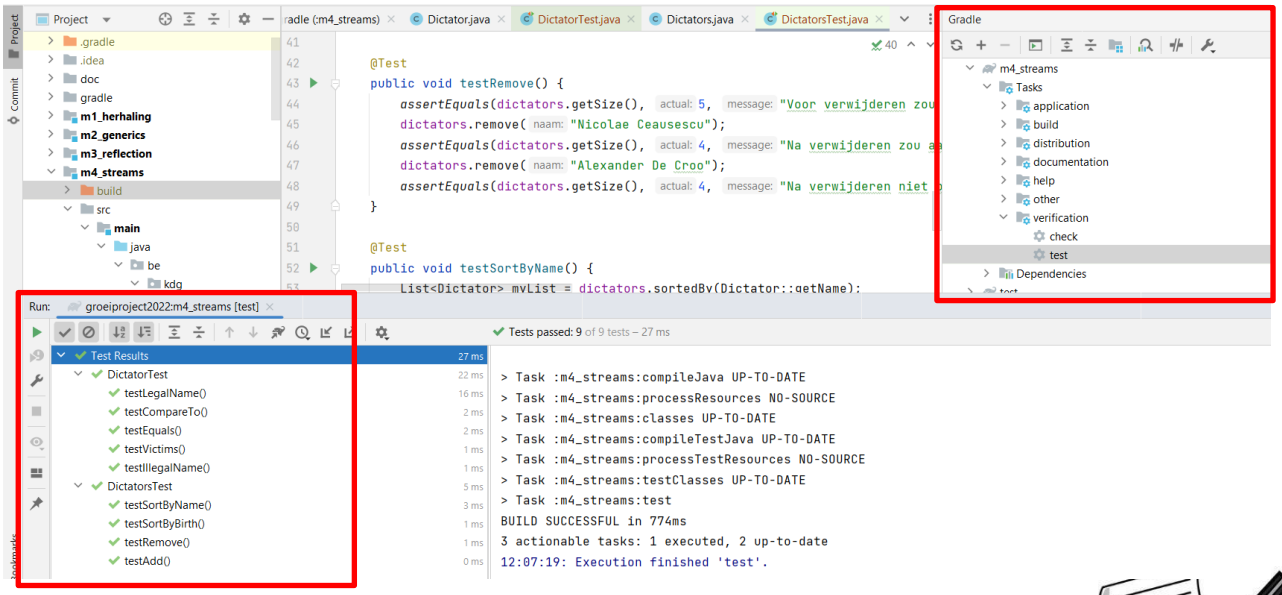
f. Voorzie nog een methode om het sorteren op een ander criterium te testen. Maak een array met daarin de 5 basisobjecten in de juiste volgorde. Gebruik dan `assertArrayEquals` om te vergelijken met de gesorteerde list (`myList.toArray()`)

g. Run en controleer of je groen licht krijgt voor alle 4 testen.

2.3. Run nu alle testklassen vanuit je terminal venster met **gradlew test**.
Ga op zoek naar het testreport (onder build>reports>tests>...) en controleer de informatie:



Run de tests ook eens via je **gradle-venster** rechts: kies de juiste module en daaronder*: verification > test :*





3. LOGGING

Maak nu pas een nieuwe module aan met de naam: `m5_logging` en voorzie een `build.gradle` met `mainClassName = 'Demo_5'`.
Kopieer van module `m1_herhaling` de hele `model` package (met o.a. de oorspronkelijke basis- en multiklasse) naar de nieuwe module.

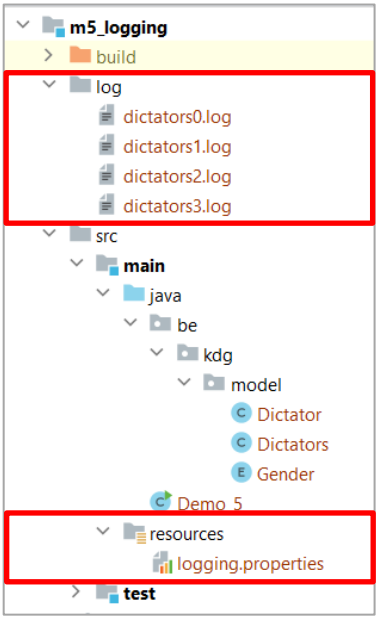
3.1. Voorzie in de basisklasse een extra `Logger`-attribuut. In alle setters waar je een IllegalArgumentException gooit als de meegegeven parameter foutief is, VERVANG je de throw door een **logging** te doen met niveau SEVERE. De boodschap moet (naast tekst) ook de foute waarde en de naam van het object bevatten.
Bijvoorbeeld: `"Geboortedatum 2078-12-18 ligt niet in het verleden voor Jozef Stalin"`

3.2. Maak een nieuwe klasse `Demo_5` met een `main`. Maak daar een aantal foutieve basisobjecten aan om de logging te testen.

Voorbeeld resultaat (logging en gewone output kunnen gemengd op de console getoond worden):

```
BUILD SUCCESSFUL in 372ms
2 actionable tasks: 2 executed
okt. 18, 2022 12:23:40 P.M. be.kdg.model.Dictator setBirth
SEVERE: Verkeerde geboortedatum voor Koen Schram
okt. 18, 2022 12:23:40 P.M. be.kdg.model.Dictator setRegime
SEVERE: Verkeerd regime voor dictator Jozef Stalin
okt. 18, 2022 12:23:40 P.M. be.kdg.model.Dictator setDuration
SEVERE: Verkeerde duur (0 moet groter zijn dan 0) voor Nicolae Ceausescu
12:23:40: Execution finished ':m5_logging:Demo_5.main()'.
```

3.3. Voeg in de multiklasse logging van niveau FINER toe die registreert welke objecten toegevoegd worden aan of verwijderd uit de TreeSet.



3.4. Maak onder de module `m5_logging` een directory met de naam `log`. De bedoeling is dat in deze map alle logbestanden terecht Komen →

3.5. Voorzie een `logging.properties` bestand met daarin de configuratie voor de logging. Zet dat bestand in de `resources` map van deze module, dan kan je er gemakkelijk naar verwijzen. Zorg ervoor dat alle boodschappen van niveau FINER of hoger naar een logbestand in de log-map geschreven worden. Een log bestand mag maximum 10KB groot zijn en je houdt enkel de 4 laatste bestanden bij.

3.6. Vanuit de main in `Demo_5` zorg je ervoor dat het configuratiebestand `logging.properties` geladen wordt. Dat kan bijvoorbeeld als volgt:

```
InputStream inputStream =
        Demo_5.class.getResourceAsStream("logging.properties");
LogManager.getLogManager().readConfiguration(inputStream)
```

Test nu in de main het toevoegen en verwijderen in de multiklasse uit.
Voer de code meerdere keren uit (**best runnen vanuit gradle anders vindt hij het juiste pad niet**) en bekijk de aangemaakte logfiles; dat zouden er max 4 mogen zijn.
Controleer de inhoud van de logfiles: zowel de FINER als de SEVERE meldingen moeten hierin gelogd worden.
Voorbeeld van een logfile:

```
<?xml version="1.0" encoding="windows-1252" standalone="no"?>
<!DOCTYPE log SYSTEM "logger.dtd">
<log>
<record>
  <date>2022-10-18T10:32:34.365279700Z</date>
  <millis>1666089154365</millis>
  <nanos>279700</nanos>
  <sequence>0</sequence>
  <logger>be.kdg.model.Dictator</logger>
  <level>SEVERE</level>
  <class>be.kdg.model.Dictator</class>
  <method>setBirth</method>
  <thread>1</thread>
  <message>Verkeerde geboortedatum voor Koen Schram</message>
</record>
<record>
  <date>2022-10-18T10:32:34.394109900Z</date>
```

3.7. Maak een nieuwe package `logging` en schrijf daarin een klasse `SmallLogFormatter`, die overerft van de abstracte klasse `java.util.logging.Formatter`. Doe een override van de `format`-methode. Haal uit het `LogRecord` de noodzakelijke informatie om volgende geformatteerde String op te bouwen:

`date time` `Level:` *`LEVEL`* `melding:` *"boodschap"*

Voorbeeld:
"2022-10-18 13:18:06.379053500 Level: SEVERE melding: "Verkeerde geboortedatum voor Koen Schram"

- Om uit het LogRecord een `LocalDateTime` te halen gebruik je:
  `LocalDateTime ldt = LocalDateTime.ofInstant(record.getInstant(), ZoneId.systemDefault());`
- Om uit de record de log boodschap te halen gebruik je:
  `MessageFormat.format(record.getMessage(), record.getParameters())`
- Configureer de ConsoleHandler in `logging.properties`, zodat deze jouw formatter gebruikt.

Voorbeeld van gewenste console logging:

```
BUILD SUCCESSFUL in 335ms
3 actionable tasks: 2 executed, 1 up-to-date
2022-10-18 13:18:06.379053500 Level: SEVERE melding: "Verkeerde geboortedatum voor Koen Schram"
2022-10-18 13:18:06.404293 Level: SEVERE melding: "Verkeerd regime voor dictator Jozef Stalin"
2022-10-18 13:18:06.411189200 Level: SEVERE melding: "Verkeerde duur (0 moet groter zijn dan 0) voor Nicolae Ceausescu"
13:18:06: Execution finished 'run'.
```

Een hele boterham; proficiat!

Zorg dat je alles veilig op git zet 😊

